Midterm exam — Version B — ANSWER KEY

TC1037 Algorithmic Thinking · Group 510 · Tecnológico de Monterrey, Campus Querétaro

Friday 23 October 2026 · 17:35–18:55 · Room 4206

Name: _____ **Matrícula:** _____

80 minutes · 100 points · individual · on paper. No computer, phone, notes or calculator. Write in pen or dark pencil; code must be legible and indented. **Part I** (30 pts): write *exactly* what Python prints — every line, in order. **Part II** (70 pts): write the program. Suggested time: 15 minutes for Part I, 65 for Part II.

Question:	1	2	3	4	5	6	7	8	9	10	11	Total
Points:	5	5	5	5	5	5	23	23	3	20	1	100
Score:												

Part I — Read and predict **(30 points)**

1. **Arithmetic operators.** What does Python print? [5]

```
print(9 + 7 // 2 * 3 - 10 % 3 ** 2)
```

Solution:
```
17
```

2. **Boolean operators.** What does Python print? [5]

```
print(not 5 < 4 and 3 == 3.0 or 2 > 2)
```

Solution:
```
True
```

3. **Conditional structure.** What does Python print? [5]

```
x = 14
y = 5
if x % y < 3:
    print("A", x + y)
elif x > y and x % 2 == 0:
    print("B", x - y)
else:
    print("C")
print("end")
```

Solution:
```
B 9
end
```

4. **Functions with a `for` loop.** What does Python print? [5]

```python
def g(a, b):
    for i in range(1, b):
        a = a * i
    return a

def main():
    a = 4
    b = 2
    c = g(b, a)
    print(a, b, c)

main()
```

Solution: `4 2 12`

5. **The while loop.** What does Python print? [5]

```python
n = 2
total = 0
while n < 30:
    total = total + n
    n = n * 2
print(n, total)
```

Solution: `32 30`

6. **The for loop.** What does Python print? [5]

```python
for i in range(12, 0, -4):
    if i % 3 == 0:
        print(i)
    else:
        print(-i)
```

Solution: `12`
`-8`
`-4`

Part II — Write the program **(70 points)**

Write **one** Python program with the following parts. All `input()` and all validation happen in `main()`; the two functions **return** their result and never print or read. **Function 7 must use a `while` loop and function 8 a `for` loop** — each is worth 23 points. Write each function's purpose in a one-line comment above it, and under each loop a one-line comment saying why it stops (`while`) or how many times it runs (`for`).

7. **`while loop`** The function `count_even_digits(n)` receives a non-negative integer `n` and **returns** how many of its digits are even (0, 2, 4, 6, 8). Use a `while` loop with `%` and `//`; no strings, no lists. Careful: the number `0` has one digit, and it is even. [23]

   Examples: `count_even_digits(2468)` → `4` `count_even_digits(1357)` → `0` `count_even_digits(1020)` → `3` `count_even_digits(0)` → `1`

8. **`for loop`** The function `sum_even_squares(n)` receives an integer $n \geq 1$ and **returns** the sum of the squares of the even numbers from 1 to n, *including* n if it is even: $2^2 + 4^2 + \cdots$. Use a `for` loop. [23]

   Examples: `sum_even_squares(1)` → `0` `sum_even_squares(4)` → `20` `sum_even_squares(5)` → `20` `sum_even_squares(6)` → `56`

9. The function `menu()` that displays: `1. count_even_digits 2. sum_even_squares 3. Exit` [3]

10. The function `main()` that, inside a `while` loop: shows the menu, reads the option, and [20]
    - option 1: reads n (must be 0 or more); if any value is out of range displays a clear message, otherwise calls `count_even_digits` and displays the result with a label;
    - option 2: reads n (must be 1 or more); same rule, calling `sum_even_squares`;
    - option 3: displays `Goodbye.` and the program ends;
    - any other option: displays `Invalid option.` and the program ends.

    Use `if`/`elif`/`else` to choose the option.

11. The last line of the script calls `main()`. [1]

Technical requirements — 2 points off for each one missing: meaningful variable names in Python style · a clear prompt for every input · readable, labelled output.

Reference solution

```python
# Reference solution — Midterm exam — Version B

def count_even_digits(n):
    if n == 0:
        return 1
    count = 0
    while n > 0:
        if (n % 10) % 2 == 0:
            count = count + 1
        n = n // 10
    return count


def sum_even_squares(n):
```

```
    total = 0
    for i in range(1, n + 1):
        if i % 2 == 0:
            total = total + i ** 2
    return total


def menu():
    print("1. count_even_digits")
    print("2. sum_even_squares")
    print("3. Exit")


def main():
    keep_going = True
    while keep_going:
        menu()
        option = int(input("Option: "))
        if option == 1:
            n = int(input("Number n (0 or more): "))
            if n < 0:
                print("Invalid input: n must be 0 or more.")
            else:
                print("Even digits:", count_even_digits(n))
        elif option == 2:
            n = int(input("n (1 or more): "))
            if n < 1:
                print("Invalid input: n must be 1 or more.")
            else:
                print("Sum of the even squares:", sum_even_squares(n))
        elif option == 3:
            print("Goodbye.")
            keep_going = False
        else:
            print("Invalid option.")
            keep_going = False


main()
```

Part I Q4: `g(b, a)` crosses the arguments (`a`=2, `b`=4 inside); `range(1, 4)` is 1, 2, 3 → $2 \cdot 1 \cdot 2 \cdot 3 = 12$; main unchanged → `4 2 12`. Common wrong answers: `4 2 4` (no swap: `range(1, 2)` is just 1) and `4 2 48` (`range(1, 4)` read as 1–4).

What to look for. `count_even_digits(0)` must return 1: a plain `while n > 0` returns 0 — the trap. `sum_even_squares`: `range(1, n)` drops n when it is even (`(6)` gives 20, not 56). Accept `range(2, n + 1, 2)` without the `if`.

How Part II is graded (the academia's rubric)

Each part of the program (the two functions, `menu`, `main`, the call) is scored on this scale, as a percentage of its points:

Excelente 100%	**Muy bien 85%**	**Regular 70%**	**Deficiente 50%**	**Muy deficiente 20%**	**Sin evidencia 0%**
No syntax errors; works correctly, including the boundary cases in the examples.	Correct approach; minimal logic or syntax slips.	At most two syntax errors and/or small logic errors; the approach is closely related to what was asked.	Approach loosely related, or related but with more than two syntax errors.	Code unrelated to the problem.	No code.

Part I: 5 points per question, all or nothing — every line of the output, in order. Technical requirements: -2 each, from Part II.